MSD Module Specification

API and Algorithm Standard for Time-Ordered AtomisticFrameCollection Analysis

mdstats

2026-07-22

1. Purpose

This document specifies the mean-square displacement (MSD) module for the `mdstats` package. The module consumes a normalized `AtomisticFrameCollection` object and computes displacement statistics for selected atoms.

The design supports two distinct uses:

1. **Stationary diffusion analysis** using an average over many time origins.
2. **Nonstationary trajectory diagnosis** using displacement from one fixed reference frame, useful for melting, structural collapse, equilibration, and other transitions.

The module returns raw displacement observables. Diffusion fitting, local-slope analysis, and uncertainty estimation are separate post-processing operations.

2. Dependency on `AtomisticFrameCollection`

MSD is a temporal observable. The input must satisfy

```
collection.frame_semantics is FrameSemantics.TRAJECTORY
```

An independent ensemble is rejected even when it stores source step or time labels, because frame adjacency and displacement continuity are not physical. The implementation calls `require_trajectory()`, `require_minimum_frames(2)`, and `require_time_axis()` before constructing displacements.

The MSD module assumes the parser and preprocessor have already enforced the following invariants:

- The number of atoms is constant across all frames.
- Atomic ordering is persistent across all frames.
- Atomic species and masses are constant.
- Frame times are stored explicitly in ps.
- Cell matrices are stored for every frame in ASE row-vector convention.
- Fractional positions are continuous and unwrapped.
- Arrays use `float64` unless otherwise stated.

The required trajectory fields are conceptually:

```
class AtomisticFrameCollection:
    frame_semantics: FrameSemantics
    frame_ids: NDArray[np.int64]                    # (T,)

    atomic_numbers: NDArray[np.int32]              # (N,)
    masses: NDArray[np.float64]                    # (N,), amu
    pbc: NDArray[np.bool_]                         # (3,)

    steps: NDArray[np.int64]                       # (T,)
    times: NDArray[np.float64]                     # (T,), ps
    cells: NDArray[np.float64]                     # (T, 3, 3), A
    fractional_positions: NDArray[np.float64]      # (T, N, 3), unwrapped
```

Velocities, forces, stresses, and thermodynamic quantities are not required for MSD.

3. Coordinate convention

The three lattice vectors are rows of the cell matrix

$$H_t = \begin{pmatrix} \mathbf{a}_t^T \\ \mathbf{b}_t^T \\ \mathbf{c}_t^T \end{pmatrix}.$$

For a row-vector fractional coordinate $\tilde{\mathbf{s}}_{i,t}$, the unwrapped Cartesian coordinate is

$$\mathbf{r}_{i,t} = \tilde{\mathbf{s}}_{i,t} H_t.$$

The MSD implementation must never re-unwrap coordinates. Unwrapping belongs to the parser/preprocessor and is treated as complete before analysis begins.

3.1 Laboratory-frame coordinates

In laboratory mode,

$$\mathbf{r}_{i,t}^{\mathrm{lab}} = \tilde{\mathbf{s}}_{i,t} H_t.$$

This includes both atomic motion and homogeneous deformation of a variable cell. It is the natural convention for fixed-cell NVT trajectories.

3.2 Reference-cell coordinates

For variable-cell diffusion analysis, every frame may instead be mapped into a fixed reference cell:

$$\mathbf{r}_{i,t}^{\mathrm{ref}} = \tilde{\mathbf{s}}_{i,t} H_{\mathrm{ref}}.$$

Supported reference cells are:

- the initial cell;
- the arithmetic mean cell;
- an explicit user-provided 3×3 matrix.

Reference-cell coordinates remove homogeneous cell deformation while retaining motion in fractional space. They are often preferable for NPT diffusion and melting diagnosis.

4. MSD definitions

Let A be the selected atom set and $|A|$ its size.

4.1 Time-origin-averaged MSD

For frame lag k,

$$\mathrm{MSD}_{\mathrm{avg}}[k] = \frac{1}{N_{\mathrm{orig}}(k)|A|} \sum_{n \in \mathcal{O}_k} \sum_{i \in A} \left| \mathbf{r}_{i,n+k} - \mathbf{r}_{i,n} \right|^2,$$

where $\mathcal{O}_k$ is the selected set of valid time origins.

This definition assumes approximate stationarity. It is the preferred raw observable for equilibrium diffusion calculations.

4.2 Fixed-origin MSD

For a fixed origin frame n_0,

$$\mathrm{MSD}_{n_0}[n] = \frac{1}{|A|} \sum_{i \in A} \left| \mathbf{r}_{i,n} - \mathbf{r}_{i,n_0} \right|^2, \qquad n \geq n_0.$$

This definition is intentionally nonstationary. It preserves the temporal sequence of a phase transition instead of averaging solid, transition, and liquid regimes together.

For a melting trajectory, a common qualitative pattern is:

```
vibrational rise -> solid plateau -> transition growth -> liquid linear regime
```

The liquid regime is identified by an approximately constant slope, not by a constant MSD.

4.3 Displacement second-moment tensor

For either mode, define

$$M_{\alpha\beta} = \langle \Delta r_\alpha \Delta r_\beta \rangle.$$

The Cartesian components are

$$\mathrm{MSD}_x = M_{xx}, \qquad \mathrm{MSD}_y = M_{yy}, \qquad \mathrm{MSD}_z = M_{zz},$$

and the scalar MSD is

$$\mathrm{MSD} = \mathrm{Tr}\ M.$$

5. Drift removal

Uniform translation can create a false long-time contribution. For a drift reference set R, define either its center of geometry

$$\mathbf{c}_R(t) = \frac{1}{|R|} \sum_{j \in R} \mathbf{r}_j(t),$$

or its center of mass

$$\mathbf{c}_R(t) = \frac{\sum_{j \in R} m_j \mathbf{r}_j(t)}{\sum_{j \in R} m_j}.$$

Corrected coordinates are

$$\mathbf{r}'_i(t) = \mathbf{r}_i(t) - \mathbf{c}_R(t).$$

Drift correction is optional. The drift reference selection is independent of the measured atom selection. For example, cation motion may be measured relative to a zeolite framework.

Subtracting the center of the same mobile species removes collective translation of that species and may suppress a physically meaningful mode. The choice must therefore remain explicit.

6. Public API

```python
from __future__ import annotations

from typing import Literal, Sequence

import numpy as np
from numpy.typing import ArrayLike, NDArray


def compute_msd(
    collection: AtomisticFrameCollection,
    *,
    species: str | int | Sequence[str | int] | None = None,
    atom_indices: ArrayLike | None = None,
    mode: Literal["time_averaged", "fixed_origin"] = "time_averaged",
    origin_frame: int = 0,
    max_lag: int | None = None,
    origin_stride: int = 1,
    lag_stride: int = 1,
    coordinate_mode: Literal[
        "laboratory",
        "reference_cell",
```

```
    ] = "laboratory",
    reference_cell: Literal["initial", "mean"] | NDArray[np.float64] = "mean",
    drift_mode: Literal[
        "center_of_mass",
        "center_of_geometry",
    ] | None = None,
    drift_species: str | int | Sequence[str | int] | None = None,
    drift_atom_indices: ArrayLike | None = None,
    compute_tensor: bool = True,
    per_atom: bool = False,
    backend: Literal["auto", "direct", "fft"] = "auto",
    atom_block_size: int | None = None,
) -> MSDResult:
    ...
```

6.1 Selection rules

- `species` and `atom_indices` are mutually exclusive.
- If neither is given, all atoms are selected.
- Species may be symbols or atomic numbers.
- Explicit atom indices refer to the canonical internal atom order.
- Duplicate indices are rejected.
- Drift selection follows the same rules.
- If drift correction is requested without an explicit drift selection, all atoms are used as the drift reference.

6.2 Mode-specific arguments

For `mode="time_averaged"`:

- `origin_stride` controls spacing between time origins.
- `max_lag` defaults to `n_frames // 2`.
- `origin_frame` is ignored and should trigger no effect.
- `backend="direct"` evaluates displacements explicitly.
- `backend="fft"` requires `origin_stride == 1` and uses blocked position autocorrelations.
- `backend="auto"` selects a backend from an estimated computational-work model.
- `atom_block_size` limits the number of atoms transformed together; when it is omitted, the shared FFT planner chooses a block size from a conservative memory target.

For `mode="fixed_origin"`:

- `origin_frame` selects the reference frame.
- Output frames begin at `origin_frame`.
- `lag_stride` controls output sampling.
- `origin_stride` is ignored.
- `max_lag` limits the farthest frame from the fixed origin.
- The direct $O(TM)$ algorithm is always used; `backend="fft"` is rejected.

7. Output structure

```
from dataclasses import dataclass, field
from typing import Any


@dataclass(frozen=True, slots=True)
class MSDResult:
    lag_steps: NDArray[np.int64]        # (L,)
    lag_times: NDArray[np.float64]      # (L,), ps

    msd: NDArray[np.float64]            # (L,), A^2
    components: NDArray[np.float64]     # (L, 3), A^2
    tensor: NDArray[np.float64] | None  # (L, 3, 3), A^2
```

```
    per_atom_msd: NDArray[np.float64] | None  # (L, M), A^2
    n_origins: NDArray[np.int64]        # (L,)

    atom_indices: NDArray[np.int64]     # (M,)
    n_atoms: int

    mode: str
    coordinate_mode: str
    drift_mode: str | None
    reference_cell: NDArray[np.float64] | None

    metadata: Mapping[str, Any] = field(default_factory=dict)
    signature: DynamicsInputSignature | None = None
```

Required identities are

$$\mathrm{MSD}[k] = \sum_{\alpha=x,y,z} \mathrm{MSD}_\alpha[k] = \mathrm{Tr}\ M[k],$$

and

$$\mathrm{MSD}[0] = 0.$$

For fixed-origin mode, `n_origins` is an array of ones. Metadata records at least:

```
mode
origin frame, step, and physical time
selected atom indices
coordinate convention
reference-cell definition
origin stride and lag stride
requested and chosen backend
FFT length, FFT atom block size, and coordinate-centering convention
D0 displacement atom/origin block sizes, memory target, and peak-work estimate
drift convention and drift selection
```

8. Time-grid constraint

The first implementation requires a uniformly sampled time grid:

$$t_{n+1} - t_n = \Delta t.$$

The check should use a numerical tolerance appropriate for `float64` times. A nonuniform trajectory must be rejected because averaging by frame lag would mix unequal physical lag times.

Support for nonuniform trajectories would require time-separation binning or resampling and is outside the initial module.

9. High-level algorithm

```
AtomisticFrameCollection
    |
    +-- D0 prepare_displacement_inputs
    |      +-- validate trajectory and uniform time grid
    |      +-- resolve measured and drift-reference selections
    |      +-- construct laboratory or reference-cell coordinates
    |      +-- subtract the resolved drift trajectory
    |      +-- attach the complete DynamicsInputSignature
    |
    +-- dispatch by mode and backend
           |
           +-- time_averaged / direct
           |      +-- resolve atom/origin block plan
```

```
|      +-- iterate lag-major displacement blocks
|      +-- accumulate and normalize second moments
|
+-- time_averaged / blocked FFT
|      +-- subtract one constant position per atom
|      +-- accumulate position auto/cross spectra in atom blocks
|      +-- combine correlations with prefix-summed endpoint terms
|
+-- fixed_origin / direct
       +-- choose one reference frame
       +-- subtract it from every requested output frame
       +-- average outer products over atoms

-> MSDResult
```

10. Pseudocode

10.1 Coordinate construction

```
function construct_coordinates(trajectory, selection, mode, reference_cell):
    S <- trajectory.fractional_positions[:, selection, :]

    if mode == "laboratory":
        return einsum("tni,tij->tnj", S, trajectory.cells)

    Href <- resolve_reference_cell(reference_cell, trajectory.cells)
    return einsum("tni,ij->tnj", S, Href)
```

10.2 Time-origin-averaged direct mode

The direct backend consumes the D0 bundle and iterator specified in `_displacement_common_spec.md`.

```
bundle <- prepare_displacement_inputs(..., full Cartesian subspace)
plan <- resolve_displacement_block_plan(bundle, requested_lags,
                                        origin_stride,
                                        memory_target=256 MiB)
initialize raw component, tensor, and optional per-atom sums

for block in iter_displacement_blocks(bundle, requested_lags, plan):
    squared <- block.displacements * block.displacements
    component_sum[block.lag_index] += sum(squared over origins and atoms)
    tensor_sum[block.lag_index] += sum(delta outer delta)
    per_atom_sum[block.lag_index, block atoms] += sum(|delta|^2 over origins)

normalize once by exact origin and atom counts
```

The canonical Cartesian D0 bundle is required because `MSDResult.components` and `MSDResult.tensor` retain laboratory x, y, z axes. Projected scalar moments are derived later from the complete source result through `AnalysisSubspace`.

For a block, the tensor accumulation is

```
tensor_sum[k] += np.einsum(
    "oai,oaj->ij",
    block.displacements,
    block.displacements,
    optimize=True,
)
```

Different valid block shapes may alter floating-point reduction grouping but must agree with the pre-D0 direct algebra within strict `float64` tolerance.

10.3 FFT time-origin-averaged mode

For one Cartesian coordinate,

$$\mathrm{MSD}_x(k) = \frac{\sum_{n=0}^{T-k-1} x_{n+k}^2 + \sum_{n=0}^{T-k-1} x_n^2 - 2\sum_{n=0}^{T-k-1} x_n x_{n+k}}{T-k}.$$

The final term is a positive-lag linear autocorrelation and is evaluated by zero-padded FFT. The two squared-coordinate sums are obtained from cumulative sums. For the full tensor,

$$M_{\alpha\beta}(k) = \frac{A_{\alpha\beta}^{\mathrm{early}}(k) + A_{\alpha\beta}^{\mathrm{late}}(k) - C_{\alpha\beta}(k) - C_{\beta\alpha}(k)}{(T-k)|A|}.$$

Before transformation, one constant position is subtracted from each atom,

$$\mathbf{r}_i'(t) = \mathbf{r}_i(t) - \mathbf{r}_i(0),$$

which leaves every displacement unchanged while reducing cancellation for large unwrapped coordinates. Atoms are processed in memory-limited blocks. The FFT backend requires all time origins (`origin_stride == 1`).

```
center positions by atom
plan zero-padded FFT length and atom block size
for each atom block:
    transform x, y, z position series
    accumulate self auto/cross spectra
    accumulate coordinate-product time series
    optionally retain per-atom scalar MSD
invert accumulated spectra
combine correlation and endpoint sums
divide by atom count and exact origin counts
```

10.4 Fixed-origin mode

```
reference <- positions[origin_frame]
frames <- origin_frame + requested_lags

delta <- positions[frames] - reference

tensor <- mean(delta outer delta over atoms)
components <- diagonal(tensor)
msd <- trace(tensor)

if per_atom:
    per_atom_msd <- |delta|^2
```

A vectorized tensor accumulation is

```
tensor = np.einsum(
    "tai,taj->tij",
    delta,
    delta,
    optimize=True,
) / delta.shape[1]
```

11. Complexity

For L requested lags, T frames, and M selected atoms, the direct time-averaged algorithm scales approximately as

$$O(LTM).$$

The blocked FFT backend scales approximately as

$$O(MT \log T),$$

while fixed-origin MSD remains $O(TM)$. The prepared Cartesian coordinate array requires approximately

$$24TM \text{ bytes}$$

in `float64`. D0 additionally bounds direct displacement work in both atom and origin dimensions. For subspace rank d, its conservative block estimate is

$$8A_bO_b(9 + 2d) \text{ bytes} .$$

The direct MSD path uses a 256 MiB target. FFT work arrays remain bounded by `atom_block_size`; per-atom output still requires the final $L \times M$ array. The direct implementation remains the numerical reference and is preferred for short calculations or sparse time origins.

12. Validation and errors

The implementation must reject:

- fewer than two frames;
- a nonuniform or nonmonotonic time grid;
- an empty selection;
- out-of-range or duplicate atom indices;
- simultaneous `species` and `atom_indices` arguments;
- invalid species symbols or atomic numbers;
- `origin_frame` outside the trajectory;
- `max_lag < 0` or `max_lag >= n_frames` after mode-specific adjustment;
- nonpositive `origin_stride` or `lag_stride`;
- an invalid backend or nonpositive `atom_block_size`;
- `backend="fft"` with fixed-origin mode or `origin_stride != 1`;
- a singular, nonfinite, or incorrectly shaped reference cell;
- a drift reference group with zero total mass;
- nonfinite trajectory coordinate or cell data.

Tiny negative FFT roundoff is clamped only within a scale-dependent tolerance. Materially negative diagonal moments issue a numerical warning rather than being silently hidden.

Warnings should be issued when:

- laboratory coordinates are used with strongly varying cells;
- long lags have very few valid origins;
- drift removal uses the same small mobile subset being measured;
- fixed-origin mode is used for quantitative equilibrium diffusion fitting.

13. Edge cases and interpretation

13.1 Wrapped coordinates

The MSD module assumes positions are already unwrapped. Applying the algorithm to wrapped positions creates artificial jumps and invalid results.

13.2 NPT trajectories

Laboratory-frame MSD includes affine expansion, contraction, shear, and cell rotation. Reference-cell MSD removes homogeneous deformation but is not a unique physical observable. Both conventions should be labeled explicitly in plots and exported data.

13.3 Melting and other transitions

Time-origin averaging can hide nonstationarity. Fixed-origin MSD is useful for locating the transition, but it is sensitive to the selected origin frame and to collective drift. After identifying a stationary liquid interval, compute a new time-origin-averaged MSD on that interval for diffusion analysis.

13.4 Vibrational plateau

A solid does not generally have identically zero MSD. Atomic vibrations produce an initial rise followed by a plateau. Departure from that plateau is a useful transition signal.

13.5 Finite-size and collective effects

Subtracting total center-of-mass motion is often numerically useful, but it changes collective transport. Small simulation cells may also exhibit strong finite-size correlations.

13.6 Per-atom MSD

Per-atom MSD is useful for heterogeneous hopping, but individual curves are noisy and are not independent samples when atoms interact strongly.

14. Deferred post-processing

The following functions belong outside `compute_msd`:

```
def fit_diffusion_coefficient(...):
    """Fit the linear diffusive regime with an explicit time window."""


def compute_local_msd_slope(...):
    """Estimate d(MSD)/dt or D_eff(t) with controlled smoothing."""


def estimate_msd_uncertainty(...):
    """Estimate statistical uncertainty using block or bootstrap methods."""
```

For isotropic diffusion in three dimensions,

$$D = \frac{1}{6}\frac{d\ \mathrm{MSD}}{dt}.$$

The fitting interval must be selected explicitly; the MSD routine must not silently infer or fit it.

15. Minimum test suite

The implementation should include deterministic tests for:

1. A static structure: MSD is zero.
2. Uniform translation: known quadratic fixed-origin MSD.
3. Constant-velocity particles: exact analytic displacement.
4. Random walk: approximately linear long-time averaged MSD.
5. Tensor consistency: scalar equals trace and sum of components.
6. Fixed-origin versus time-averaged semantics.
7. Species and explicit-index selections.
8. Center-of-mass and center-of-geometry drift removal.
9. Variable-cell laboratory versus reference-cell coordinates.
10. Per-atom output consistency with the atom-averaged result.
11. Rejection of nonuniform times and malformed arguments.
12. D0 deterministic block ordering and complete sample coverage.
13. Direct-MSD parity under multiple atom and origin block sizes.
14. Hard D0 memory-target compliance and immutable block outputs.

16. Final implementation boundary

The implemented MSD module provides:

- explicit `time_averaged` and `fixed_origin` modes;
- species or canonical-index selection;
- laboratory and reference-cell coordinates;
- optional center-of-mass or center-of-geometry drift removal;
- scalar, Cartesian-component, and full-tensor MSD;
- optional per-atom MSD;
- D0 atom/origin-blocked direct and atom-blocked FFT backends for time-averaged MSD;

- work-based automatic backend selection;
- shared FFT planning and positive-lag correlation primitives with `vacf.py`;
- strict validation and reproducibility metadata.

The direct implementation remains the numerical reference. Fixed-origin MSD and sparse-origin time averaging remain direct because the standard FFT correlation estimator does not reproduce those sampling rules.

The module does not perform automatic diffusion fitting, local-slope smoothing, uncertainty estimation, VACF integration, or irregular-time resampling. Those operations remain separate post-processing modules.

H0 dynamics-contract integration

Every computed `MSDResult` carries a complete `DynamicsInputSignature` containing the exact trajectory fingerprint, frame sequence and times, measured atoms, coordinate/reference-cell semantics, drift mode and exact drift-reference atoms, and the source three-dimensional analysis basis.

All result arrays are owned and read-only. Metadata is recursively immutable. A supplied signature must agree with the result's measured atoms, coordinate mode, drift mode, reference cell, and full source 3D subspace. `origin_stride`, `lag_stride`, `max_lag`, and `atom_block_size` reject booleans; `compute_tensor` and `per_atom` require actual booleans.

MSD/VACF diffusion comparison is fail-closed: unsigned legacy results are not comparable, and the complete signatures must agree after applying the VACF estimate's physical subspace to the MSD. Canonical axis subsets may use stored components; a rotated projection requires the full MSD tensor. The fit divisor is $2d$, where d is the stored subspace rank, never an independently supplied interpretation. See `_dynamics_common_spec.md` and `diffusion_estimation_spec.md`.

D0 shared-displacement integration

Release `0.19.81a0` moves all measured-selection, coordinate, reference-cell, drift, and signature construction into `prepare_displacement_inputs()`. The direct time-origin-averaged estimator consumes immutable `DisplacementBlock` objects in deterministic lag/origin/atom order. The FFT estimator consumes the same prepared Cartesian positions but intentionally retains independent correlation algebra.

The direct result metadata records:

```
displacement_common_stage = "D0"
displacement_atom_block_size
displacement_origin_block_size
displacement_memory_target_bytes
displacement_estimated_peak_work_bytes
```

The public `atom_block_size` argument remains the FFT block control in this release. Direct D0 block-size controls remain internal because they do not change the scientific estimator and are primarily exercised by infrastructure and regression tests.

See `_displacement_common_spec.md` for the normative bundle, planner, iterator, validation, memory, ordering, and immutability contracts.